

General Remarks:

- Make sure that we can follow the steps of your calculations. Do not combine too many small steps in one and leave enough intermediate steps. For the exercises where you need to calculate concrete inverses and eigen-values you don't need to calculate these manually.
- Visual explanation of the Newton-Rhapson method: [here](#)

1 Exercise 1 - Optimization Methods

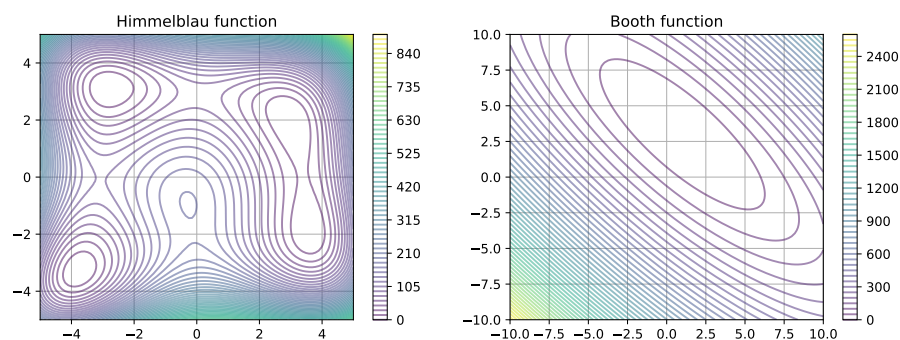
Exercise 1:

In this exercise you have to minimize different functions, two test functions are provided, one has to be defined by you.

- booth(xy)
- himmelblau(xy)
- yourfunction(xy)

The booth and himmelblau functions¹ are defined in the limits -10 to 10 and -5 to 5 respectively.

Optimize (minimize) using the gradient-descent-, Newton-Rhapson- and BFGS-method for different starting points. A couple of test starting points are provided in the file, but also choose your own starting points.



In more detail:

- (1) implement the optimization procedures and perform the analysis as described below, for each:
 - (1 Point) gradient descent
 - (1 Point) Newton Rhapson method
 - (2 Points) BFGS² (You can also implement L-BFGS if you prefer)
- (2) minimize the functions for different starting points

¹Test functions for optimization

²BFGS algorithm

- (3) plot the losses (in this case the evaluation of the function at each intermediate solution)
- (4) plot the trajectories over the contour plots.

The functions are written using jax so you can get the derivative and hessian for ‘func’ using ‘jax.jacobian(func)’ and ‘jax.hessian(func)’, both of which return callables.

The BFGS algorithm proceeds as follows (assuming column vectors):

for function f and initial guess x_0 of the optimum, an initial guess of (a) the hessian estimate B_0 , or the (b) inverse of the hessian estimate B_0^{-1} :

- (a) $B_0 = I$ or (b) $B_0^{-1} = I$
- for $k \in [0, \dots]$
 - (a) solve $B_k \mathbf{p}_k = -\nabla f(\mathbf{x}_k)$ for $\mathbf{p}_k$
 - (b) $\mathbf{p}_k = -B_k^{-1} \nabla f(\mathbf{x}_k)$
 - (*) $\alpha_k = \arg \min_{\alpha} f(\mathbf{x}_k + \alpha \mathbf{p}_k)$
 - (*) $\mathbf{s}_k = \alpha_k \mathbf{p}_k$
 - (*) $\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{s}_k$
 - (*) $\mathbf{y}_k = \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k)$
 - (a) $B_{k+1} = B_k + \frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} - \frac{B_k \mathbf{s}_k \mathbf{s}_k^T B_k^T}{\mathbf{s}_k^T B_k \mathbf{s}_k}$
 - (b) $B_{k+1}^{-1} = B_k^{-1} + \frac{(\mathbf{s}_k^T \mathbf{y}_k + \mathbf{y}_k^T B_k^{-1} \mathbf{y}_k)(\mathbf{s}_k \mathbf{s}_k^T)}{(\mathbf{s}_k^T \mathbf{y}_k)^2} - \frac{B_k^{-1} \mathbf{y}_k \mathbf{s}_k^T + \mathbf{s}_k \mathbf{y}_k^T B_k^{-1}}{\mathbf{s}_k^T \mathbf{y}_k}$

2 Exercise 2 - LQR

Exercise 2:

In this exercise we take a brief look at the infinite-horizon [Linear-Quadratic-Regulator](#).

Assume a discrete-time linear system described by:

$$x_{k+1} = Gx_k + Hu_k$$

and a cost function defined by:

$$J = \sum_{k=0}^{\infty} \left(x_k^T Q x_k + u_k^T R u_k + 2x_k^T N u_k \right)$$

and an optimal control sequence given by:

$$u_k = -Kx_k$$

$$K = (R + H^T P H)^{-1} (H^T P G + N^T)$$

and P is the unique positive definite solution to the discrete time [algebraic Riccati equation](#) (DARE):

$$P = G^T P G - (G^T P H + N) (R + H^T P H)^{-1} (H^T P G + N^T) + Q$$

. This can be also written as:

$$P = \mathcal{G}^T P \mathcal{G} - \mathcal{G}^T P H (R + H^T P H)^{-1} H^T P \mathcal{G} + \mathcal{Q}$$

with:

$$\mathcal{G} = G - H R^{-1} N^T \quad \mathcal{Q} = Q - N R^{-1} N^T$$

In this exercise we will use a toolbox to solve the DARE and use the resulting control gain K to control the pendulum. The controller assumes the goal to be at $x_r = 0$ and thus defines a quadratic cost on the state $x_k^T Q x_k$ that is zero when $x_k = 0$ and a quadratic cost on the control input $u_k^T R u_k$. Usually N is not used ($= 0$). Assume R and Q to be diagonal. The state space of the pendulum consists of the angle and the angular velocity.

- Note that the LQR makes three assumptions that do not hold in our case: infinite time (our horizon is limited), infinite control input (we have box constraints, that is maximum and minimum control inputs) and infinite state space (both the angle and the velocity are bounded).
- Note that the formulation of the LQR above does only contain $Gx + Hu$ and is missing the affine term $+c$. This may or may not be a problem, e.g. when $c = 0$.
- The larger you set Q , the faster the controller should approach the goal.
- The larger you set R , the smaller the actions that the controller should choose.
- Your task is to control the pendulum and move and keep it at a specific angle. You have to implement this for different angles: $0, \frac{\pi}{2}, -\frac{\pi}{2}$
- You are provided with the skeleton code for initialization and rollouts.
- The linearization is provided and the same as in the previous exercise.

! Start with the goal position $[0, 0]$.

! Write the controller that moves the pendulum based on the control gain K .

! Tune Q and R to achieve the pendulum moving to the goal.

! Plot the quadratic costs over time, separately for distance and control input, and as the sum.

! Plot the state space transitions over time.

! Also repeat this for the angles $\frac{\pi}{2}$ and $-\frac{\pi}{2}$ as goals.

! Do you need a change of variables? For which angles (and why!) is $c = 0$ or $c \neq 0$? What is the effect? (Physical reason)

! Note that $H(u + u') = Hu + Hu'$. If necessary, and possible, use that to compensate for $c \neq 0$. (the pseudo-inverse is available as `np.linalg.pinv`). Show the difference with and compensation for $c \neq 0$ where necessary.

! Demonstrate the controller in action.

! What happens when you try the goal position $[0, 0.1]$?

- (1) (1 points) Tuned controller and demonstrations for $[0, 0]$
- (2) (1 points) Tuned controller and demonstrations for the $\frac{\pi}{2}$ and $-\frac{\pi}{2}$, with and without compensation for $c \neq 0$ where necessary.
- (3) (1 points) Explain what the goal $[0, 1.0]$ means and what is happening.

To initialise the pendulum and goal:

```
1 x_initial = jnp.array([-onp.pi, 0.])
2 env = JaxPendulum(initial_state=x_initial)
3 x_0 = env.reset()
4 x_goal = jnp.array([0., 0.])
```

Autodiff and linear step function code is provided:

```
1 def linearise_autodiff(f_step, x_prime=jnp.zeros((2,)), u_prime=jnp.zeros((1,))):
2     G = jax.jacobian(f_step, 0)(x_prime, u_prime)
3     H = jax.jacobian(f_step, 1)(x_prime, u_prime)
4     c = f_step(x_prime, u_prime) - (G @ x_prime + H @ u_prime)
5     return G, H, c
6
7 def make_F_linear(G, H, c):
8     def linear_F_step(x, u):
9         return G @ x + H @ u.reshape(-1) + c
10    return linear_F_step
```

The state of the environment is held as an internal variable, to allow easier access to the dynamics, the step function is also provided as a stateless function `env.F_step`. To linearise around the goal and solve the DARE:

```
1 G, H, c = linearise_autodiff(env.F_step, x_goal)
2
3 R = jnp.zeros((1,1)) # you need to fix this
4 Q = jnp.zeros((2, 2)) # and this.
5 C = onp.zeros_like(G) # C, D are not used, so zeros here are fine.
6 D = onp.zeros_like(H)
7 # use the control toolbox to create a state-space system:
8 ss = control.ss(G, H, C, D, dt=env.dt)
9 # solve the DARE:
10 K, S, E = control.lqr(ss, Q, R) # this gives us the K!
```

You can find the limits of the control input and state space as:

```
1 env.action_space.low # u_min
2 env.action_space.high # u_max
3 env.observation_space.low # x_min
4 env.observation_space.high # x_max
```

3 Exercise 3 - QP

Similar to the LQR exercise we are going to control the Pendulum.

The exercise consists of three parts:

- (1) Part1 (2 Points): Writing the `setup_problem` function.

```
1 def setup_problem( horizon, x0, x_r, dim_x, dim_u, hG, hH, hc,
2                    xmin, xmax, umin, umax, Q, R, Q_final=None, goal_constraint=True
3                ):
4     ...
5     return P, q, C, h, A, b
```

Note that instead of G, H, c we have hG, hH, hc which is a sequence of length `horizon` (N), and in each timestep i you should use the respective G_i from `hG[i]` (etc.). This exercise essentially requires you to build the matrices similar to Exercise Sheet 3.

The goal constraint is similar to the equality constraint for x_0 and should, if set to true, add a constraint to equal $x_N = x_r$.

- (2) Part2 (1 Point): Tune the cost in the receding horizon model-predictive-control (MPC) task and describe the `receding_horizon_MPC` function. This function is already provided to you, but you have to explain it in the PS.

Receding-horizon MPC roughly works as follows: the model is used to estimate a plan (= a sequence of control inputs/actions). The model is assumed to be

imperfect, thus: if the whole plan (which was estimated based on the model) would be executed as is, errors (due to the difference between the model and the real behavior and/or sensing errors) would accumulate and compound. To circumvent the accumulation of errors, only the first step of the plan is used. After executing this first step, a new, state is reached. Using this state and the model, a new plan is estimated and the first step of this plan is executed. The planning horizon moves (recedes) at every step.

```
1  # this are untuned parameters.
2  R = onp.eye(1)
3  Q = onp.diag(onp.array([1.0, 1.0]))
4  Q_final = onp.diag(onp.array([1.0, 1.0]))
5  goal_constraint = False
6  ...
7  def receding_horizon_MPC( env, dim_x, dim_u, x_r, R, Q, horizon, steps,
8                          umin=None, umax=None, xmin=None, xmax=None, Q_final=None,
9                          goal_constraint=False, render=True, ):
10     ...
11     return trajectory, control_sequence
12
```

- (3) Part3 (2 Points): In the model-predictive-control case above, we use the same linearized dynamics for each time step. These linearized dynamics are derived from linearizing around a fixed state (e.g. the initial or the goal state). But, as we move to different states, linearizing around those states is likely to yield different linearizations. Using a fixed linearization is suboptimal and executing the resulting control sequence would not solve the problem under the given non-linear (“real”) dynamics. In this exercise, we try to refine that.

Follow the instructions in the code:

- a) linearize around the initial point and use the same dynamics for all timesteps.
- b) plan a trajectory using these dynamics. (i.e. see the MPC example on how).
- c) iteratively refine the dynamics, i.e. building new hG , hH , hc :
 - i. start linearising around x_0 as usual
 - ii. for the other points $x_i, \dots, x_N$ in the previously planned trajectory hx , calculate $x_i = F_{\text{step}}(hx[i-1], hu[i-1])$ and linearise around x_i
 - iii. optimize again for hx and hu .
 - iv. compare how much the dynamics and the planned trajectory has changed in each iteration (i.e. to see if it converges)
- d) Rollout (open-loop) the control-sequence hu and render the result. Does this work better than the initial hu ?

3.1 Quadratic Program Solution

The solution to setting up the quadratic program will be handed out separately after Sheet 3.